

# Agents, Architecture & Automation



**SoftUni Team**  
Technical Trainers



**SoftUni**



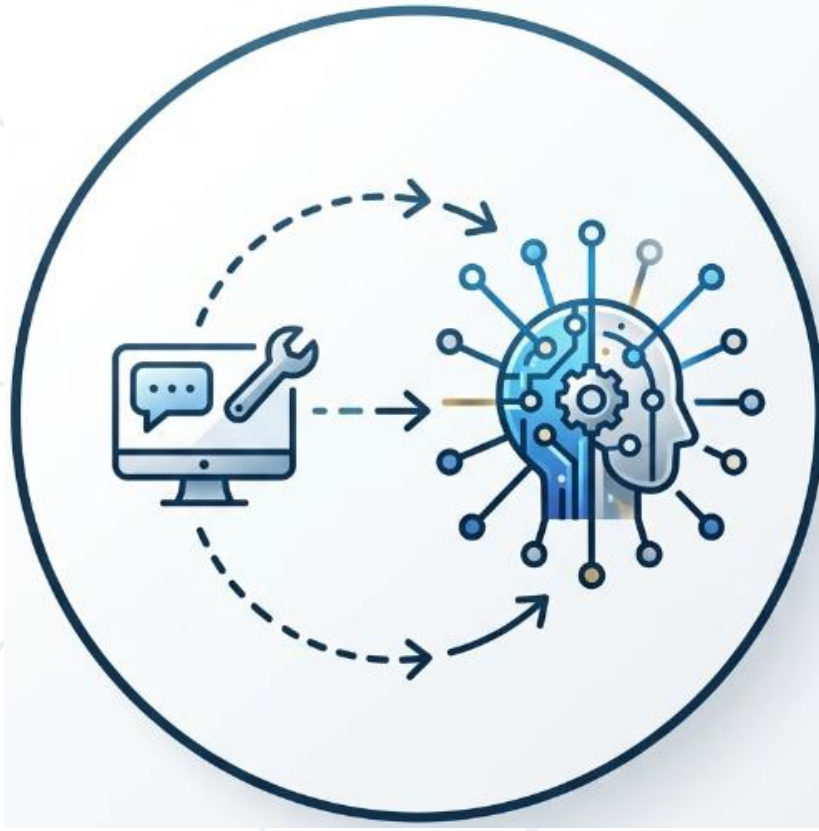
**Software University**

<https://softuni.org>

# Table of Contents

1. From **Assistants** to **Agents**: The evolution of IDE AI
2. Agent **Architecture**: How AI plans and remembers
3. Empowering AI: Leveraging **Tools** and Custom **Skills**
4. Divide and Conquer: The role of **Subagents**
5. Advanced Customization: **Instructions**, **Files**, and **Hooks**
6. Model Context Protocol (**MCP**): Standardizing AI context
7. Real-World **Workflows**: TDD, Web Testing, and Debugging
8. Security, Monitoring, and the **Future** of AI Coding





# **From Assistants to Autonomous Agents**

# Recap: The Limits of Standard AI Chat

- Requires constant **human** prompting
- **Limited** awareness of broader project scope
- **Stateless** interactions (forgets context easily)
- Can **suggest** code, but **cannot execute** actions

- Definition: Systems that pursue complex **goals** autonomously
- Move from "Answer my question" to "**Solve** this problem"
- Capable of **planning, executing, and observing results**
- Applicable across modern AI IDE extensions

- LLMs are no longer just **text generators**
- Used as the "brain" to **orchestrate complex tasks**
- Routing capabilities: Deciding which **tool** or **agent** to use
- Evaluating **outcomes** and correcting course **autonomously**

# The Agent Architecture (Conceptual)

- **User Intent** (Goal definition)
- **Planning** (Breaking down the goal)
- **Memory** (Storing context)
- **Tools & Action** (Executing commands)
- **Observation** (Reviewing the output)



# How Agents "Think" – Planning and Memory



- **Multi-step** reasoning process
- **Decomposing** large tasks into sub-tasks
- Dynamic adaptation: **Changing** the **plan** if a step fails
- **Transparency**: Showing the user the execution plan

- **Short-term** memory: Current chat session context
- **Long-term** memory: Remembering past interactions and project rules
- Maintaining continuity across different coding sessions
- Context Windows vs. Persistent Storage

- **Restoring** previous states of an AI interaction
- **Reviewing** code edits at different stages of generation
- **Safe** rollback **mechanisms** during agent execution
- **Debugging** chat interactions (Chat Debug View)

- **Cloud** Agents: High compute, advanced reasoning models
- **Local** Agents: Privacy-focused, lower latency, offline capabilities
- **Hybrid** approaches: Routing sensitive data locally
- Trade-offs: Security vs. Capability



**Empowering Agents – Tools and Skills**

# What are Agent Tools?

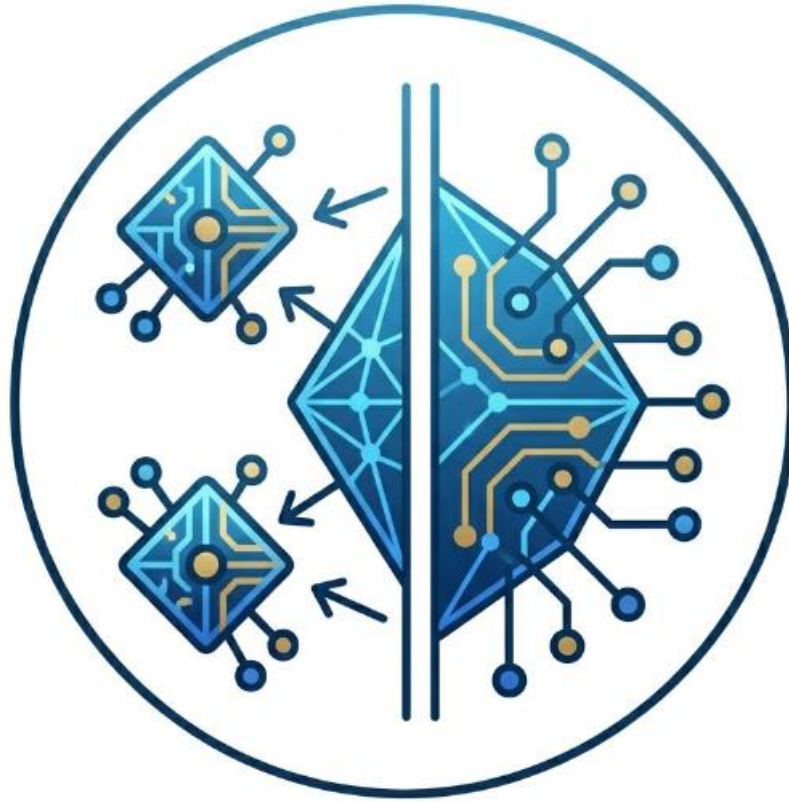
- Giving the AI "hands" to **manipulate** the environment
- Moving beyond text generation to action execution
- **Examples**: Reading files, running terminal commands, searching web
- The **boundary** between Read-Only and Write/Execute tools

- **Specialized, repeatable** capabilities assigned to an agent
- Customizing how an agent **approaches** a specific domain
- **Skills** vs. **Tools**: Skills use tools to accomplish specific domain tasks
- **Enhancing** generic models with **domain-specific** expertise

- Defining the skill's **trigger** (when it should be used)
- Providing **specific instructions** and parameters
- Connecting the **skill** to available workspace tools
- **Testing** and **iterating** on skill accuracy



- The "**Human-in-the-loop**" concept
- **Reviewing** code edits before applying
- **Explicit** user **permission** for terminal execution
- **Preventing destructive actions** (Security Best Practices)



**Divide and Conquer – Subagents**

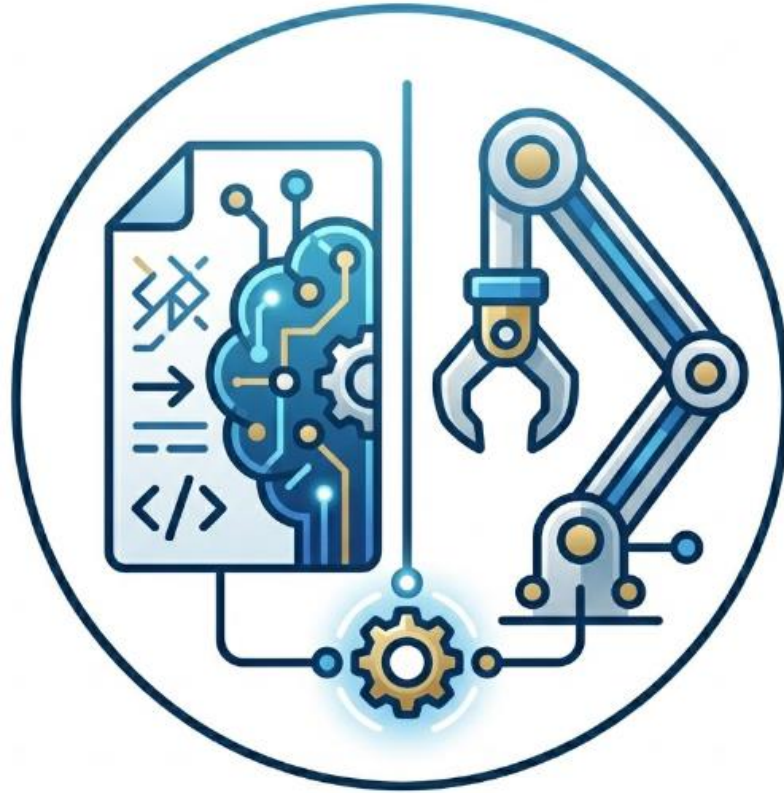
- One monolithic agent becomes **inefficient** for complex workspaces
- The concept of "**Separation of Concerns**" in AI
- **Specialized agents** for specialized **tasks**
- **Hierarchical** execution models

# How Subagents Work

- Primary Agent (**Orchestrator**) receives the user prompt
- Orchestrator analyzes intent and **delegates** to **Subagents**
- Subagents execute **specific tasks** (e.g., Testing, Searching)
- Results are synthesized and returned to the user

- **Workspace/Codebase** Agent (Searches local files)
- Terminal/CLI Agent (Executes shell commands)
- **Browser** Agent (Searches documentation or tests web apps)
- Third-Party Agents (Integrations with external services)

- Passing context between **Orchestrator** and **Subagent**
- Sharing **state** and **memory** artifacts
- Handling **conflicts** or errors from subagents
- The future of multi-agent ecosystems in the IDE



# **Advanced Customization – Instructions & Hooks**

- Moving **beyond default** IDE settings
- Adapting the AI to your specific project architecture
- Custom Instructions vs. Prompt Files vs. Custom Agents
- **Workspace-level** vs. **User-level** configurations



- **System prompts** that guide agent behavior
- Defining coding **standards** and **frameworks**
- Formatting **rules** (e.g., "Always write tests for new functions")
- **Avoiding repetitive** prompt engineering

- **Storing complex prompts** as code (.md or .prompt files)
- **Version-controlling** your AI instructions
- **Sharing** prompt templates across the development team
- **Reusability** for common tasks (Code review, Refactoring)

- **Event-driven** AI interventions
- **Triggering** AI actions based on IDE lifecycle events
- **Automating workflows** without manual chat initiation
- Bridging the gap between manual AI use and **automation**

- **On-Save** Hook: Auto-format or suggest optimizations
- **Pre-Commit** Hook: Generate commit messages automatically
- **On-Test-Failure** Hook: Auto-diagnose and suggest fixes
- **Post-Build** Hook: Analyze build errors



# **Model Context Protocol (MCP) – The Game Changer**

# The Integration Challenge

- AI needs access to **external**, fragmented data
- **Custom API integrations** for every tool are unscalable
- The need for a **universal standard** in AI context gathering

# What is MCP (Model Context Protocol)?

- An open **standard connecting** AI models to data sources
- **Replaces custom integrations** with a unified protocol
- Similar to LSP (Language Server Protocol), but for AI context
- **Separates** the AI Client (IDE) from the Data Server

- **MCP Hosts:** The IDE extension initiating the request
- **MCP Clients:** The protocol layer managing communication
- **MCP Servers:** The data providers (Databases, APIs, Local files)
- Standardized JSON-RPC communication



# What can MCP Servers provide?

- **Resources**: Static data (File contents, API responses)
- **Prompts**: Pre-defined templates for specific tasks
- **Tools**: Executable functions the AI can call via the server
- Seamless integration directly into the AI Agent's context



# Real-World Agent Workflows

- How to **feed the right data** to your agents
- Using Chat Artifacts Panel for **context visualization**
- Avoiding "Context Pollution" (too much irrelevant data)
- **Best practices** for referencing workspace context

- Providing the agent with **requirements**
- Agent generates failing tests first
- Agent writes implementation code to pass tests
- **Iterative refinement** based on test output

- Using browser-specific subagents
- Rendering UI components for **AI inspection**
- AI interacting with the **DOM**
- Automating **end-to-end testing** with natural language

- Feeding **stack traces** to the agent
- Agent using tools to read related source files
- Formulating hypotheses and **running local tests**
- Explaining the **root cause** and **proposing fixes**

- Tracking **agent performance** and execution times
- Reviewing **tool usage logs**
- Identifying infinite loops or **failed planning states**
- Troubleshooting common agent **failures**

- **Preventing** prompt injection in shared workspaces
- Understanding **what data is sent** to Cloud Agents
- **Restricting** filesystem access for Tools and Skills
- Enterprise policy controls



- **Transition** from "Code Completion" to "Software Engineering Agents"
- **Deepening integration** with MCP ecosystems
- **Increased autonomy** vs. Developer oversight
- The **changing role** of the software developer

- **Agents over Chat:** AI is moving from passive text generation to autonomous task execution.
- **Context is King:** Memory, Prompts, and MCP servers dictate the quality of AI output.
- **Modularity:** Subagents and specialized Skills allow for highly targeted problem-solving.
- **Automation:** Hooks and custom instructions integrate AI seamlessly into standard development lifecycles.
- **Human-in-the-Loop:** Despite increased autonomy, reviewing edits and ensuring secure tool execution remains essential.



- Software University – High-Quality Education, Profession and Job for Software Developers
  - [softuni.bg](https://softuni.bg)
- Software University Foundation
  - [softuni.foundation](https://softuni.foundation)
- Software University @ Facebook
  - [facebook.com/SoftwareUniversity](https://facebook.com/SoftwareUniversity)



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg>
- © Software University – <https://softuni.bg>

